PROGRAMMING 512 ASSIGNMENT

RICHFIELD
richfield.ac.za

Name and Surname: Thashin Marimuthu

Student ITS No: 402603393

Qualification: BSc IT **Year of Study:** 1 **Semester:** 2

Assignment due date: 2026/10/23 **Date submitted:** 2026/10/23

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents
Introduction
Main Body (with relevant subheadings)
Conclusion
References

Submit the assignment in PDF format on Moodle. Use the specified cover page provided.
Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:
I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism

Student Signature:  **Date:** 2026/10/23

Contents

Introduction:.....3
Code:.....4
Admin GUI: 4
Lecturer GUI:.....16
Login GUI: 27
Operator GUI:.....32
Administrator: 37
Booking: 38
Lecturer:.....39
Operator: 40
Resource: 41
User: 42
Durban: 43
Johannesburg:.....44
Pretoria:.....45
Database: 46
Reports:.....64
Main: 66
Conclusion:.....67
GitHub Link: 68
Reference List:.....69

Introduction:

The Smart Campus Resource Booking System is a desktop application developed using Python, Tkinter, and SQLite to assist higher education institutions in managing campus resources efficiently. The system provides a centralized platform where lecturers can book resources, campus administrators can manage resources, and system operators can monitor overall system performance across multiple campuses.

The application was designed using Object-Oriented Programming (OOP) principles, including inheritance and encapsulation, to create a structured and maintainable solution. The system supports multiple campuses with unique booking policies, resource management, booking validation, reporting, and persistent data storage through SQLite. The project demonstrates the practical application of programming concepts such as classes, functions, lists, dictionaries, sets, graphical user interfaces (GUI), database integration, and role-based access control.

Code:

Admin GUI:

```
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox
from database import get_all_resources
from database import delete_resource
from database import add_resource
from database import get_campus_bookings
from database import update_resource
from database import get_resource_usage_by_campus

class AdminGUI:

  def __init__(self):

    self.window = tk.Toplevel()

    self.window.title(
      "Campus Administrator"
    )

    self.window.geometry("700x500")

    tk.Label(
      self.window,
      text="Resource ID"
```

```
).pack()

self.resource_id = tk.Entry(
   self.window
)

self.resource_id.pack()

tk.Label(
   self.window,
   text="Resource Name"
).pack()

self.resource_name = tk.Entry(
   self.window
)

self.resource_name.pack()

tk.Label(
   self.window,
   text="Campus"
).pack()

self.campus = ttk.Combobox(
   self.window,
   values=[
      "Johannesburg",
```

```
        "Pretoria",
        "Durban"
    ]
)

self.campus.pack()

tk.Button(
    self.window,
    text="Add Resource",
    command=self.add_resource
).pack(pady=20)

tk.Button(
    self.window,
    text="Refresh Resources",
    command=self.view_resources
).pack(pady=10)

tk.Button(
    self.window,
    text="Delete Selected Resource",
    command=self.remove_resource
).pack(pady=10)

self.resource_list = tk.Listbox(
    self.window,
    width=80
```

```
)

self.resource_list.pack(pady=20)

tk.Button(
    self.window,
    text="View Campus Bookings",
    command=self.view_campus_bookings
).pack(pady=10)

self.booking_list = tk.Listbox(
    self.window,
    width=90
)

self.booking_list.pack(pady=10)

tk.Button(
    self.window,
    text="View Usage Trends",
    command=self.view_usage_trends
).pack(pady=10)

self.usage_list = tk.Listbox(
    self.window,
    width=90
)
```

```
    self.usage_list.pack(pady=10)

    tk.Label(
        self.window,
        text="New Resource Name"
    ).pack()

    self.new_name = tk.Entry(
        self.window
    )

    self.new_name.pack()

    tk.Button(
        self.window,
        text="Update Resource",
        command=self.edit_resource
    ).pack(pady=10)

def add_resource(self):

    if (
        self.resource_id.get() == ""
        or
        self.resource_name.get() == ""
        or
        self.campus.get() == ""
    ):
```

```
    messagebox.showerror(
        "Error",
        "Complete all fields"
    )

    return

success = add_resource(
    self.resource_id.get(),
    self.resource_name.get(),
    self.campus.get()
)

if success:

    messagebox.showinfo(
        "Success",
        "Resource Added"
    )

    self.view_resources()

else:

    messagebox.showerror(
        "Error",
        "Duplicate Resource ID"
```

```
        )

    def view_resources(self):

        self.resource_list.delete(
            0,
            tk.END
        )

        resources = get_all_resources()

        for resource in resources:

            resource_text = (
                f"{resource[0]} | "
                f"{resource[1]} | "
                f"{resource[2]}"
            )

            self.resource_list.insert(
                tk.END,
                resource_text
            )
    def remove_resource(self):

        selected = self.resource_list.curselection()

        if not selected:
```

```
            messagebox.showerror(
                "Error",
                "Select a resource"
            )

            return

        resource = self.resource_list.get(
            selected[0]
        )

        resource_id = resource.split("|")[0].strip()

        delete_resource(resource_id)

        messagebox.showinfo(
            "Success",
            "Resource Deleted"
        )

        self.view_resources()

    def view_campus_bookings(self):

        self.booking_list.delete(
            0,
            tk.END
```

```
    )

    campus = self.campus.get()

    bookings = get_campus_bookings(
        campus
    )

    for booking in bookings:

        booking_text = (
            f"Lecturer: {booking[0]} | "
            f"Resource: {booking[1]} | "
            f"Date: {booking[2]} | "
            f"Time: {booking[3]} | "
            f"Duration: {booking[4]}"
        )

        self.booking_list.insert(
            tk.END,
            booking_text
        )

def edit_resource(self):

    selected = self.resource_list.curselection()

    if not selected:
```

```
    messagebox.showerror(
        "Error",
        "Select Resource"
    )

    return

resource = self.resource_list.get(
    selected[0]
)

resource_id = (
    resource
    .split("|")[0]
    .strip()
)

update_resource(
    resource_id,
    self.new_name.get()
)

messagebox.showinfo(
    "Success",
    "Resource Updated"
)
```

```
    self.view_resources()

def view_usage_trends(self):

    self.usage_list.delete(
        0,
        tk.END
    )

    campus = self.campus.get()

    usage = get_resource_usage_by_campus(
        campus
    )

    if not usage:

        self.usage_list.insert(
            tk.END,
            "No booking data available"
        )

        return

    for item in usage:

        self.usage_list.insert(
            tk.END,
```

```
    f"Resource: {item[0]} | Total Bookings: {item[1]}"
)
```

Lecturer GUI:

```
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox
from database import get_user_bookings
from database import cancel_booking
from database import get_booking_rule
from database import get_resources
from database import create_booking
from database import get_available_resources


class LecturerGUI:

  def __init__(self, username):

    self.username = username

    self.window = tk.Toplevel()

    self.window.title("Lecturer Dashboard")
    self.window.geometry("800x600")

    tk.Label(
      self.window,
      text=f"Welcome {username}",
      font=("Arial", 16)
    ).pack(pady=20)
```

```
tk.Label(
    self.window,
    text="Select Campus"
).pack()

self.campus_box = ttk.Combobox(
    self.window,
    values=[
        "Johannesburg",
        "Pretoria",
        "Durban"
    ]
)

self.campus_box.pack()

self.policy_label = tk.Label(
    self.window,
    text=""
)

self.policy_label.pack()

tk.Button(
    self.window,
    text="Load Resources",
    command=self.load_resources
).pack(pady=10)
```

```
self.resource_box = ttk.Combobox(
    self.window
)

self.resource_box.pack()

self.availability_label = tk.Label(
    self.window,
    text="Available Resources"
)

self.availability_label.pack()

tk.Label(
    self.window,
    text="Booking Date (DD/MM/YYYY)"
).pack()

self.date_entry = tk.Entry(
    self.window
)

self.date_entry.pack()

tk.Label(
    self.window,
    text="Start Time"
```

```
).pack()

self.time_entry = tk.Entry(
    self.window
)

self.time_entry.pack()

tk.Label(
    self.window,
    text="Duration"
).pack()

self.duration_entry = tk.Entry(
    self.window
)

self.duration_entry.pack()

tk.Button(
    self.window,
    text="Book Resource",
    command=self.book_resource
).pack(pady=20)

tk.Button(
    self.window,
    text="View My Bookings",
```

```
        command=self.view_bookings
    ).pack(pady=10)

    tk.Button(
        self.window,
        text="Cancel Selected Booking",
        command=self.delete_booking
    ).pack(pady=10)

    self.booking_list = tk.Listbox(
        self.window,
        width=80
    )

    self.booking_list.pack(pady=20)

def view_bookings(self):

    self.booking_list.delete(
     0,
        tk.END
    )

    bookings = get_user_bookings(
        self.username
    )

    for booking in bookings:
```

```
        booking_text = (
            f"{booking[0]} | "
            f"{booking[1]} | "
            f"{booking[2]} | "
            f"{booking[3]} | "
            f"{booking[4]} | "
            f"{booking[5]} hour(s)"
        )

        self.booking_list.insert(
            tk.END,
            booking_text
        )

    def delete_booking(self):

        selected = self.booking_list.curselection()

        if not selected:

            messagebox.showerror(
                "Error",
                "Select a booking first"
            )

            return
```

```
    booking = self.booking_list.get(
        selected[0]
    )

    booking_id = booking.split("|")[0].strip()

    cancel_booking(
        booking_id
    )

    messagebox.showinfo(
        "Success",
        "Booking Cancelled"
    )

    self.view_bookings()

def load_resources(self):

    self.update_policy()

    campus = self.campus_box.get()

    resources = get_available_resources(
        campus
    )

    self.resource_box["values"] = resources
```

```
def book_resource(self):

  try:

    campus = self.campus_box.get()

    duration = int(
      self.duration_entry.get()
    )

    max_duration = get_booking_rule(
      campus
    )

    if duration > max_duration:

      messagebox.showerror(
        "Policy Rule",
        f"Maximum booking duration is {max_duration} hours"
      )

      return

    if self.resource_box.get() == "":

      messagebox.showerror(
        "Error",
```

```
            "Select a resource"
        )

        return

    success = create_booking(
        self.username,
        campus,
        self.resource_box.get(),
        self.date_entry.get(),
        self.time_entry.get(),
        duration
    )

    if success:

        messagebox.showinfo(
            "Success",
            "Booking Created"
        )

        self.load_resources()

    else:

        messagebox.showerror(
            "Error",
            "Resource Unavailable"
```

```
            )

    except ValueError:

        messagebox.showerror(
            "Error",
            "Duration must be a number"
        )

def update_policy(self):

    campus = self.campus_box.get()

    if campus == "Johannesburg":

        self.policy_label.config(
            text="Policy: Maximum 4 Hours"
        )

    elif campus == "Pretoria":

        self.policy_label.config(
            text="Policy: Maximum 3 Hours"
        )

    elif campus == "Durban":

        self.policy_label.config(
```

```
    text="Policy: Maximum 2 Hours"
)
```

Login GUI:

```
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox

from database import register_user
from database import login_user
from gui.admin_gui import AdminGUI
from gui.lecturer_gui import LecturerGUI
from gui.operator_gui import OperatorGUI

root = tk.Tk()

root.title("Smart Campus Resource Booking System")
root.geometry("700x500")

def register():

    username = username_entry.get()
    password = password_entry.get()
    role = role_box.get()

    if username == "" or password == "":
        messagebox.showerror(
            "Error",
            "All fields required"
        )
        return
```

```
    success = register_user(
        username,
        password,
        role
    )

    if success:
        messagebox.showinfo(
            "Success",
            "Registration Successful"
        )
    else:
        messagebox.showerror(
            "Error",
            "User already exists"
        )

def login():

    username = username_entry.get()
    password = password_entry.get()

    result =login_user(
        username,
        password
    )
```

```
    if result:

        role = result[0]

        if role == "Administrator":

            AdminGUI()

        elif role == "Lecturer":

            LecturerGUI(username)

        elif role == "Operator":

            OperatorGUI()

        else:

            messagebox.showerror(
                "Error",
                "Unknown Role"
            )

title = tk.Label(
    root,
    text="Smart Campus Resource Booking System",
    font=("Arial",16,"bold")
)
```

```
title.pack(pady=20)

tk.Label(
    root,
    text="Username"
).pack()

username_entry = tk.Entry(root)
username_entry.pack()

tk.Label(
    root,
    text="Password"
).pack()

password_entry = tk.Entry(
    root,
    show="*"
)

password_entry.pack()

tk.Label(
    root,
    text="Role"
).pack()
```

```
role_box = ttk.Combobox(
    root,
    values=[
        "Lecturer",
        "Administrator",
        "Operator"
    ]
)

role_box.pack()

tk.Button(
    root,
    text="Register",
    command=register
).pack(pady=10)

tk.Button(
    root,
    text="Login",
    command=login
).pack()

root.mainloop()
```

Operator GUI:

```
import tkinter as tk

from database import get_all_bookings
from database import get_total_bookings_per_campus
from database import get_most_used_resources
from database import get_average_duration
from database import get_total_users
from database import get_total_resources
from database import get_total_bookings

class OperatorGUI:

  def __init__(self):

    self.window = tk.Toplevel()

    self.window.title(
      "System Operator Dashboard"
    )

    self.window.geometry("900x600")

    tk.Button(
      self.window,
      text="Generate Reports",
      command=self.generate_report
    ).pack(pady=20)
```

```
    self.report_box = tk.Text(
        self.window,
        width=100,
        height=25
    )

    self.report_box.pack()

def generate_report(self):

    self.report_box.delete(
        "1.0",
        tk.END
    )

    self.report_box.insert(
        tk.END,
        "SMART CAMPUS RESOURCE BOOKING SYSTEM\n"
    )

    self.report_box.insert(
        tk.END,
        "====================================\n\n"
    )

    self.report_box.insert(
        tk.END,
```

```
    f"Total Users: {get_total_users()}\n"
)

self.report_box.insert(
    tk.END,
    f"Total Resources: {get_total_resources()}\n"
)

self.report_box.insert(
    tk.END,
    f"Total Bookings: {get_total_bookings()}\n\n"
)

campus_data = get_total_bookings_per_campus()

self.report_box.insert(
    tk.END,
    "TOTAL BOOKINGS PER CAMPUS\n\n"
)

for row in campus_data:

    self.report_box.insert(
        tk.END,
        f"{row[0]} : {row[1]} bookings\n"
    )

self.report_box.insert(
```

```
    tk.END,
    "\nMOST USED RESOURCES\n\n"
)

resources = get_most_used_resources()

for row in resources:

    self.report_box.insert(
        tk.END,
        f"{row[0]} : {row[1]} bookings\n"
    )

self.report_box.insert(
    tk.END,
    "\nAVERAGE BOOKING DURATION\n\n"
)

average = get_average_duration()

self.report_box.insert(
    tk.END,
    f"{average[0]}\n"
)

self.report_box.insert(
    tk.END,
    "\nALL BOOKINGS\n\n"
```

```
)

bookings = get_all_bookings()

for booking in bookings:

    self.report_box.insert(
        tk.END,
        f"{booking}\n"
    )
```

Administrator:

```
from models.user import User


class Administrator(User):


    def __init__(self, username, password):
        super().__init__(
            username,
            password,
            "Administrator"
        )
```

Booking:

```
    class Booking:


  def __init__(
    self,
    username,
    campus,
    resource_id,
    booking_date,
    start_time,
    duration
  ):
    self.username = username
    self.campus = campus
    self.resource_id = resource_id
    self.booking_date = booking_date
    self.start_time = start_time
    self.duration = duration
```

Lecturer:

```
from models.user import User


class Lecturer(User):


    def __init__(self, username, password):
        super().__init__(
            username,
            password,
            "Lecturer"
        )
```

Operator:

```
from models.user import User


class Operator(User):


    def __init__(self, username, password):
        super().__init__(
            username,
            password,
            "Operator"
        )
```

Resource:

```
    class Resource:


  def __init__(
    self,
    resource_id,
    resource_name,
    campus
  ):
    self.resource_id = resource_id
    self.resource_name = resource_name
    self.campus = campus
```

User:

```
class User:


    def __init__(self, username, password, role):
        self.__username = username
        self.__password = password
        self.__role = role


    def get_username(self):
        return self.__username


    def get_password(self):
        return self.__password


    def get_role(self):
        return self.__role
```

Durban:

MAX_DURATION = 2

START_HOUR = 9

END_HOUR = 15

Johannesburg:

MAX_DURATION = 4

START_HOUR = 8

END_HOUR = 17

Pretoria:

MAX_DURATION = 3

START_HOUR = 7

END_HOUR = 16

Database:

```
import sqlite3

DB_NAME = "campus_booking.db"

def get_connection():
    return sqlite3.connect(DB_NAME)

def create_tables():

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS users(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE,
        password TEXT,
        role TEXT
    )
    """)

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS resources(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        resource_id TEXT UNIQUE,
        resource_name TEXT,
        campus TEXT
```

```
    )
    """)

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS bookings(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT,
        campus TEXT,
        resource_id TEXT,
        booking_date TEXT,
        start_time TEXT,
        duration INTEGER
    )
    """)

    conn.commit()
    conn.close()

def register_user(username, password, role):

    conn = get_connection()
    cursor = conn.cursor()

    try:

        cursor.execute("""
        INSERT INTO users
        (username,password,role)
```

```
        VALUES(?,?,?)
        """, (username, password, role))

        conn.commit()
        return True

    except:
        return False

    finally:
        conn.close()

def login_user(username, password):

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT role
    FROM users
    WHERE username=?
    AND password=?
    """, (username, password))

    result = cursor.fetchone()

    conn.close()
```

```
    return result

def add_resource(resource_id, resource_name, campus):

    conn = get_connection()
    cursor = conn.cursor()

    try:

        cursor.execute("""
        INSERT INTO resources
        (resource_id, resource_name, campus)
        VALUES(?,?,?)
        """, (
            resource_id,
            resource_name,
            campus
        ))

        conn.commit()
        return True

    except:
        return False

    finally:
        conn.close()
```

```
def get_resources(campus):

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT *
    FROM resources
    WHERE campus=?
    """, (campus,))

    resources = cursor.fetchall()

    conn.close()

    return resources

def create_booking(
    username,
    campus,
    resource_id,
    booking_date,
    start_time,
    duration
):

    conn = get_connection()
    cursor = conn.cursor()
```

```
cursor.execute("""
SELECT *
FROM bookings
WHERE resource_id=?
AND booking_date=?
AND start_time=?
""", (
   resource_id,
   booking_date,
   start_time
))

overlap = cursor.fetchone()

if overlap:

   conn.close()
   return False

cursor.execute("""
INSERT INTO bookings
(
   username,
   campus,
   resource_id,
   booking_date,
   start_time,
```

```
        duration
    )
    VALUES(?,?,?,?,?,?)
    """, (
        username,
        campus,
        resource_id,
        booking_date,
        start_time,
        duration
    ))

    conn.commit()
    conn.close()

    return True

def get_resources(campus):

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT resource_id
    FROM resources
    WHERE campus=?
    """, (campus,))
```

```
    resources = cursor.fetchall()

    conn.close()

    return resources

def get_user_bookings(username):

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT id,
        campus,
        resource_id,
        booking_date,
        start_time,
        duration
    FROM bookings
    WHERE username=?
    """, (username,))

    bookings = cursor.fetchall()

    conn.close()

    return bookings
```

```
def cancel_booking(booking_id):

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    DELETE FROM bookings
    WHERE id=?
    """, (booking_id,))

    conn.commit()
    conn.close()

    return True

def get_all_resources():

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT resource_id,
        resource_name,
        campus
    FROM resources
    """)

    resources = cursor.fetchall()
```

```
    conn.close()

    return resources

def delete_resource(resource_id):

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    DELETE FROM resources
    WHERE resource_id=?
    """, (resource_id,))

    conn.commit()
    conn.close()

def get_all_bookings():

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT campus,
        resource_id,
        booking_date,
        duration
```

```
    FROM bookings
    """)

    data = cursor.fetchall()

    conn.close()

    return data

def get_total_bookings_per_campus():

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT campus,
        COUNT(*)
    FROM bookings
    GROUP BY campus
    """)

    data = cursor.fetchall()

    conn.close()

    return data

def get_most_used_resources():
```

```
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT resource_id,
        COUNT(*)
    FROM bookings
    GROUP BY resource_id
    ORDER BY COUNT(*) DESC
    """)

    data = cursor.fetchall()

    conn.close()

    return data

def get_average_duration():

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT AVG(duration)
    FROM bookings
    """)
```

```
    data = cursor.fetchone()

    conn.close()

    return data

def get_booking_rule(campus):

    rules = {
        "Johannesburg": 4,
        "Pretoria": 3,
        "Durban": 2
    }

    return rules.get(campus, 1)

def get_campus_bookings(campus):

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT username,
           resource_id,
           booking_date,
           start_time,
           duration
    FROM bookings
```

```
    WHERE campus = ?
    """, (campus,))

    bookings = cursor.fetchall()

    conn.close()

    return bookings

def update_resource(
    resource_id,
    new_name
):

    conn = get_connection()

    cursor = conn.cursor()

    cursor.execute("""
    UPDATE resources
    SET resource_name = ?
    WHERE resource_id = ?
    """, (
        new_name,
        resource_id
    ))

    conn.commit()
```

```
    conn.close()

def get_available_resources(campus):

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT resource_id
    FROM resources
    WHERE campus = ?
    """, (campus,))

    all_resources = cursor.fetchall()

    cursor.execute("""
    SELECT DISTINCT resource_id
    FROM bookings
    WHERE campus = ?
    """, (campus,))

    booked_resources = cursor.fetchall()

    conn.close()

    all_set = set()
```

```
    for resource in all_resources:
        all_set.add(resource[0])

    booked_set = set()

    for resource in booked_resources:
        booked_set.add(resource[0])

    available = all_set - booked_set

    return list(available)

def get_resource_usage_by_campus(campus):

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT resource_id,
           COUNT(*)
    FROM bookings
    WHERE campus = ?
    GROUP BY resource_id
    ORDER BY COUNT(*) DESC
    """, (campus,))

    data = cursor.fetchall()
```

```
    conn.close()

    return data

def get_total_users():

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT COUNT(*)
    FROM users
    """)

    result = cursor.fetchone()

    conn.close()

    return result[0]

def get_total_resources():

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT COUNT(*)
    FROM resources
```

```
    """)

    result = cursor.fetchone()

    conn.close()

    return result[0]

def get_total_bookings():

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    SELECT COUNT(*)
    FROM bookings
    """)

    result = cursor.fetchone()

    conn.close()

    return result[0]
```

Reports:

```
from database import get_total_bookings_per_campus
from database import get_most_used_resources
from database import get_average_duration

def generate_summary():

  report = []

  report.append(
    "SMART CAMPUS REPORT"
  )

  report.append(
    "\nBOOKINGS PER CAMPUS"
  )

  for row in get_total_bookings_per_campus():

    report.append(
      f"{row[0]} : {row[1]}"
    )

  report.append(
    "\nMOST USED RESOURCES"
  )

  for row in get_most_used_resources():
```

```
    report.append(
        f"{row[0]} : {row[1]}"
    )

average = get_average_duration()

report.append(
    f"\nAverage Duration: {average[0]}"
)

return report
```

Main:

```
from database import create_tables


create_tables()


from gui.login_gui import *
```

Conclusion:

The Smart Campus Resource Booking System successfully addresses the challenges associated with managing and booking campus resources across multiple campuses. Through the implementation of role-based access control, resource management, booking functionality, campus-specific booking policies, and comprehensive reporting features, the system provides an efficient and user-friendly solution for resource allocation.

The project demonstrates a strong understanding of Python programming, Object-Oriented Programming principles, GUI development using Tkinter, and database management using SQLite. Furthermore, the use of lists, dictionaries, sets, inheritance, encapsulation, and exception handling highlights the practical application of fundamental programming concepts learned throughout the module. Overall, the system achieves its objectives by improving resource utilization, reducing scheduling conflicts, and providing valuable reporting capabilities for decision-making and campus management.

GitHub Link:

thash2007/Pro-512

Reference List:

Code, B. (2022). *Functions in python are easy* . [online] www.youtube.com. Available at: https://www.youtube.com/watch?v=89cGQjB5R4M [Accessed 18 Oct. 2026].

NeuralNine (2022). *Importing your own python modules properly*. [online] YouTube. Available at: https://www.youtube.com/watch?v=GxCXiSkm6no [Accessed 18 Oct. 2026].

Python object oriented programming full course . (2024). [Video] Directed by Bro Code. YouTube. Available at: https://www.youtube.com/watch?v=IbMDCwVm63M [Accessed 18 Oct. 2026].

Python tkinter full course for free . (2020). [Video] Directed by Bro Code. YouTube. Available at: https://www.youtube.com/watch?v=TuLxsvK4svQ [Accessed 18 Oct. 2026].

Kite (2020). *Sqlite 3 python tutorial in 5 minutes - creating database, tables and querying [2020]*. [online] www.youtube.com. Available at: https://www.youtube.com/watch?v=girsuXz0yA8 [Accessed 18 Oct. 2026].